The issues appearing during the lab section for assignment one could be divided into two parts: the practical and the conceptual.

The practical:
Due to lack of knowledge of github priviledge management, it takes a long time for me to push the code to github as the requests are frequently turned down until github CLI is installed within the virtual machine. Also initially the finished parts are not properly stored in the virtual machine which reverts to a previous snapshot when shut down, causing extra burden to reconfigure the machine.

The order of referencing header in C file is important. The statement #include “kernel/types.h” should be prioritized to inform the gcc compiler of pre-defined variable type such as uint, uint64, etc, otherwise those types are not recognizable.

The conceptual:
It takes time to farmiliarize myself with system calls and the meaning of corresponding parameters in function prototypes such as read(), write(), pipe(), close(), exit() and fork() when addressing problems in the first half.

The flow of information when adding self-defined system call, such as getppid(), is tricky. Declaration is in syscall.h with a specific number and later added to syscall.c. The full definition needs to be done in sysproc.c file. After that, we register the entry of it in usys.pl and expose it in user.h for user space process.

When addressing problem 4.4, managing pipe file descriptors proved extremely challenging, as forgetting to close the write end in the correct process could easily lead to an unexpected deadlock. Additionally, using recursion in code is not accepted by the compiler because it is likely to fall into infinite recursion. So codes written in a loop way are adopted. This brutal hands-on experience gives me an incredibly solid and intuitive understanding of inter-process communication.

The most challenging aspect in the last problem is understanding the low-level context switching logic involving swtch() as it is written in assembly to directly manipulate hardware registers and page tables and are extremely hard to grasp.

The way header files in kernel directory are structured and how each file depends on each other is also crucial to understanding the project. The file defs.h declares all function prototypes of system calls and struct types which would be defined in detail in either .c file or .S riscv assembly file. There is also an underlying control flow in trap.c which calls update_process_times() and yield() to enable time interrupter and run processes in proc[] alternatively.

Reading source code and attempts to grasp the flow control are highly time-consuming yet necessary. Despite the steep learning curve, writing this scheduler offers invaluable, hands-on insight into how the operating system kernel manages CPU resources and preemptive multitasking.